9. MICROPROGRAMARE

OBIECTIVE

CUPRINS

9. MICROPROGRAMARE 1

9.1 Noţiuni şi concepte de bază 2
9.1.1 Organizarea memoriei de control 4
9.1.2 Suportul fizic pentru păstrarea microprogramelor 7
9.1.3 Organizarea logică a instrucţiunilor 8
9.1.3.1 Codificare verticalã 8
9.1.3.2 Codificarea orizontalã 9
9.1.3.3 Codificarea minimalã 9
9.1.3.4 Codificarea cu control rezidual 10
9.1.3.5 Codificarea cu control prin adrese 11
9.1.3.6 Codificare mixtã 12
9.1.4 Implementarea microinstrucţiunilor 12

9.2 **Proiectarea unităţii de comandă microprogramată** a calculatorului didactic **15**
9.2.1 Specificarea caracteristicilor unităţii de comanda microprogramate pentru calculatorul didactic 15
9.2.2 Specificarea detaliată a structurii calculatorului didactic 16
9.2.3 Stabilirea operaţiilor elementare pentru controlul resurselor calculatorului didactic 19
9.2.4 Stabilirea variabilelor de stare (condiţiilor de test) 20
9.2.5 Stabilirea formatului microinstrucţiunilor 22
9.2.6 Proiectarea microsecvenţiatorului 24
9.2.6.1 Descrierea microprogramului 26

9.3 Codificarea minimă a unui set de microinstrucţiuni 27

9.1 Noţiuni şi concepte de bază

Din punct de vedere structural unităţile de comandă sunt de două tipuri:

- convenţionale în sensul propus de Von Newmann ;
- microprogramate conform conceptului introdus de M.Wilkes.

Conceptul de microprogramare a fost introdus de Maurice Wilkes, de la Universitatea din Cambridge, în anul 1951, ca o alternativă sistematică de proiectare a unităţilor de comandă ale calculatoarelor numerice sau, în general, a procedurilor de control asupra primitivelor funcţionale ale unui sistem numeric.

Microprogramarea poate fi considerată o tehnică de proiectare şi implementare a funcţiilor de control a sistemelor de prelucrare a datelor numerice, ca o secvenţă de semnale de control ce interpretează static sau dinamic funcţiile de prelucrare a datelor. Semnalele de comandă, necesare la un moment dat pentru controlul primitivelor funcţionale sunt organizate într-un cuvânt de control, memorat într-o memorie PROM sau RAM. Structura cuvântului de control este influenţată de semnificaţia atribuită noţiunii de microoperaţie .

Microoperaţia (μO) este o primitivă a funcţiilor de prelucrare a datelor, care reprezintă o operaţie elementară asupra unei primitive funcţionale (transfer, acţiune de înscriere sau de incrementare, activare pe magistrală, etc.), ce se desfaşoară de obicei într-o perioadă de timp (perioada de tact sau de ceas).

Microoperaţia implică existenţa unui operator, care corespunde unei unităţi funcţionale şi a operanzilor asupra cărora acţionează.

Cuvântul de control este pus în corespondenţă cu noţiunea de microinstrucţiune.

Microinstrucţiunea (μI) este un set de microoperaţii independente de date, fără conflict de resurse, care se pot executa simultan (pe perioada unei perioade de sincronizare).

În general, microinstrucţiunile sunt de două tipuri:

- microinstrucţiuni operaţionale care controlează primitivele funcţionale ale unităţii de execuţie a sistemului numeric, asigurând fluxul de informaţie şi acţiunile asupra resurselor;
- microinstrucţiune de ramificaţie (de salt) care inspectează starea primitivelor funcţionale şi asigură ramificaţia în algoritmul de control, constituind suportul pentru implementarea deciziilor.

Prin microprogram se întelege o secvenţă de microinstrucţiuni ce implementează un algoritm care descrie :

- citirea interpretarea şi execuţia unui set de instrucţiuni maşină;
- primitive ale sistemului de operare ;
- primitive ale limbajelor de programare ;
- etc.

Structura generală a unui sistem de calcul cu unitate de comandă microprogramată este prezentată în Fig. 9.1.

Primitivele funcţionale au următoarele semificaţii :

M - memoria principală a sistemului, în care se păstrează programele ca secvenţă de instrucţiuni maşină şi datele care se prelucrează ;

ML - memoria locală a sistemului, reprezentată de registrele generale de lucru accesibile sau nu utilizatorului;

UAL - unitatea aritmetică logică ;

S I/E - subsistemul de intrări / ieşiri ;

MC - memoria de control în care se păstrează microprogramul ca secvenţă de microinstrucţiuni ;

μRI - registrul de microinstrucţiuni, care păstrează microinstrucţiunea curentă ce se execută. Conţinutul său specifică toate microoperaţiile care se execută în acel moment în unitatea de execuţie ;

μS - microsecvenţiatorul, unitatea de comandă convenţională, elementară, care asigură citirea interpretarea şi execuţia microinstrucţinilor din memoria de control precum şi înlănţuirea acestora pe baza registrului de instrucţiuni maşină RI şi a stării primitivelor funcţionale.

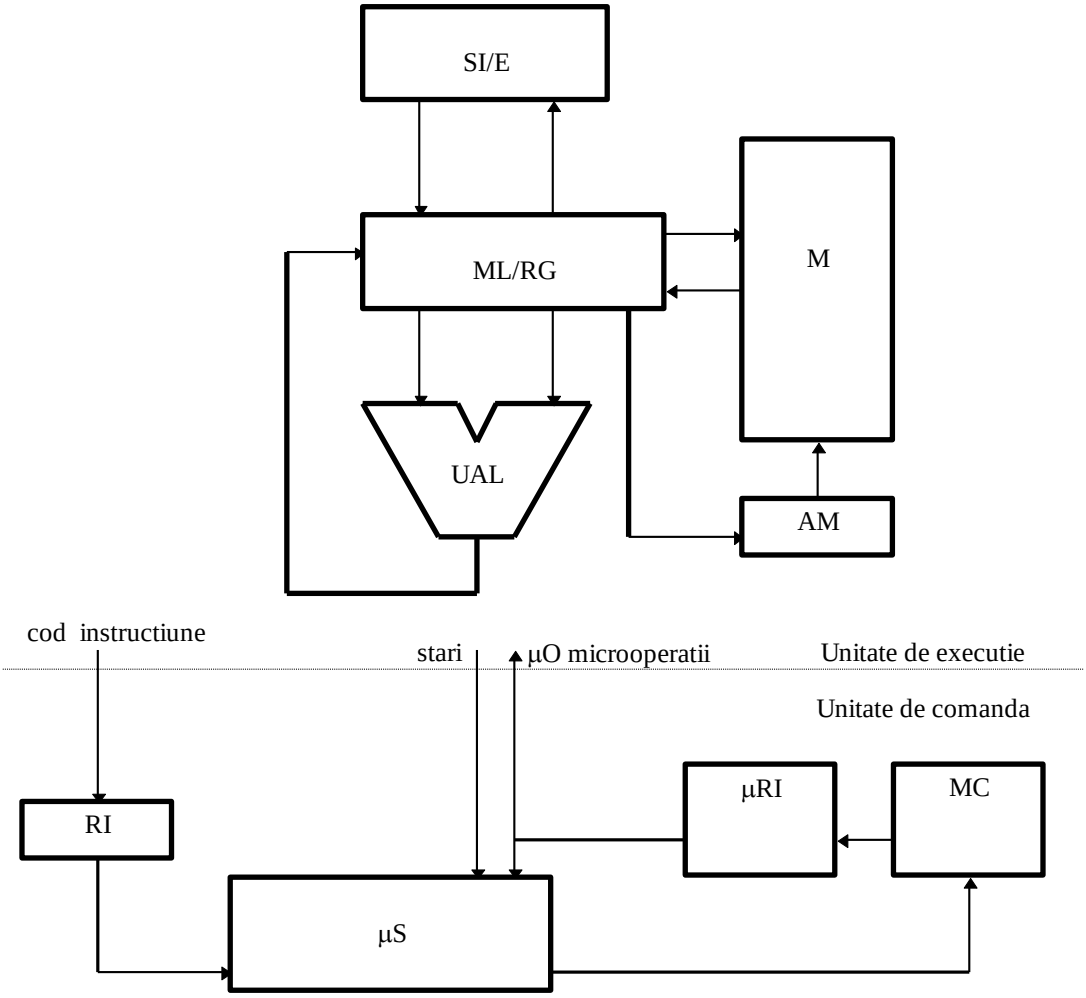


Figura ..19.19.2

Structura generala a unui sistem microprogramat

Din punctul de vedere al proiectantului de sistem, unitatea de comandă microprogramată reprezintă un calculator primar capabil să interpreteze (emuleze) orice alt pseudo calculator (calculator virtual) aflat la un nivel superior celui al calculatorului primar.

Structura simplificată prezentată în Fig. 9.1 are rolul de a scoate în evidenţă faptul că microprogramul conţine informaţiile (semnalele) de control a primitivelor funcţionale şi că acestea sunt memorate în memoria de control în acelaşi fel cum instrucţiunile maşină sunt memorate în memoria principală a sistemului.

O instrucţiune maşină este citită interpretată şi executată de o secvenţă de microinstrucţiuni care controlează şi comandă acţiunile asupra primitivelor funcţionale .

Implementarea sub formă microprogramată a unităţilor de comandă a permis dezvoltarea conceptului de emulare, care a apărut ca urmare a încercărilor de a soluţiona într-un mod eficient

problema transportabilităţii programelor, adică de a executa programele de bază şi de aplicaţii ale unui sistem pe alt sistem. Astfel, folosind aceleaşi resurse fizice, este posibil, prin diferite microprograme să se realizeze diverse structuri, care din punct de vedere funcţional, să se comporte diferit sau să corespundă unor calculatoare existente (dintr-o anumită clasă).

În general, o structură microprogramată se caracterizează prin:

- organizarea memoriei de control ;
- suportul fizic pentru păstrarea microprogramelor ;
- organizarea logică a microinstrucţiunilor ;
- implementarea microinstrucţiunilor ;

9.1.1 Organizarea memoriei de control

Organizarea memoriei de control poate fi privită din două puncte de vedere şi anume:

- relaţiei poziţionale între memoria de control MC şi memoria principală M ;
- structurii memoriei de control MC .

Folosind drept criteriu de analiză relaţia MC faţă de M se întâlnesc trei tipuri de organizare a memoriei de control :

1. *memorie de control separată de memoria principală*, atât din punct de vedere fizic cât şi punct de vedere al adresării logice, Fig. 9.2 .
   Raportul dintre viteza de lucru a memoriei de control şi cea a memoriei principale este de circa 10 în favoarea memoriei de control, Vmc >> Vm .
   O asfel de organizare întâlnim la calculatoarele din familia CORAL, I100, PDP 11, IBM 360/50, etc.
2. *memoria de control este implementată în acelaşi spaţiu* fizic şi de adresare cu cel al memoriei principale, Fig. 9.3 . Ambele au acelaşi ciclu de memorie şi sunt adresate prin aceeaşi logică . Divizarea în memorie de control şi memorie principală se poate face la nivel fizic sau la nivel logic.
   Acest tip de organizare cere ca memoria să fie suficient de rapidă pentru a fi folosită ca MC şi destul de ieftină pentru a fi folosită ca memorie principală.
   O astfel de organizare întâlnim la calculatoarele IBM 370/145; IBM 360 /25 .
3. *memoria de control este implementată separat de memoria principală* însă este încărcată din aceasta. Se utilizează un sistem de memorie ierarhică, Fig.9.4.
   Memoria MC este încărcată din memoria principală prin intermediul memoriei tampon MT.
   Acest tip de organizare este curent utilizat la calculatoarele mari cum ar fi IBM 370/145. Performanţele acestui tip de organizare depind de structura maşinii de bază şi de algoritmul de lucru cu memoria tampon.

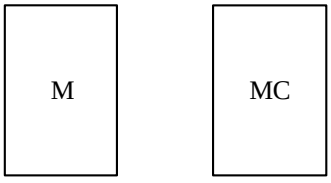


Figura 9.3
Organizarea memoriei de control

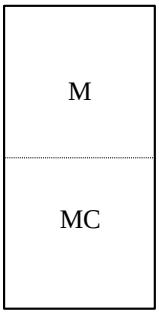


Figura 9.4
Organizarea MC în cadrul MP

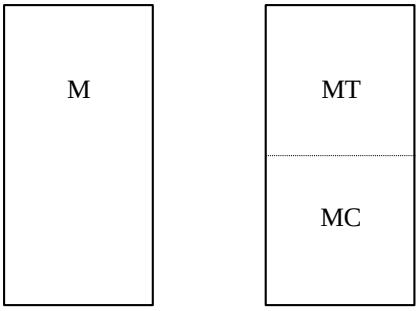


Figura 9.5
Organizarea MC separată de MP dar incarcabilă din aceasta

Din punctul de vedere al structurii memoriei de control MC aceasta poate fi organizată în mai multe moduri şi anume :

1. *memorie de control cu o microinstrucţiune pe cuvânt*, în care fiecărui cuvânt de memorie îi corespunde o singură microinstrucţiune, Fig. 9.5. Citirea unei microinstrucţiuni presupune un singur acces la MC.

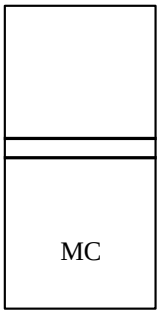


Figura 9.6
MC cu microinstructiune pe cuvânt

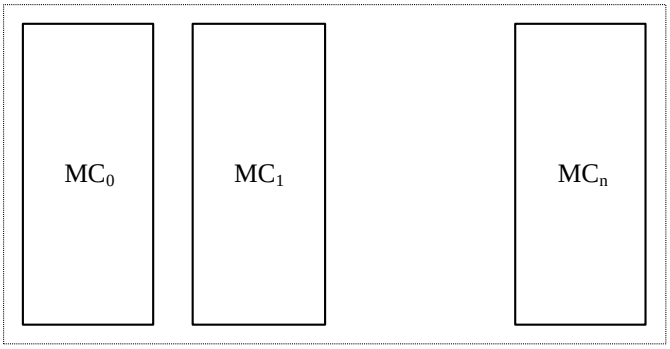


Figura 9.7
MC cu organizare pe pagini

2. *memorie de control cu organizare pe pagini*, Fig. 9.6.
   Unei adrese din memoria de control i se asociază mai multe microinstrucţiuni, din pagini diferite. în acest fel se asigură la nivel de microprogram execuţia unei microinstrucţiuni de tip CASE, care introduce facilitatea de decizii multiple .
   O adresă de MC va adresa aceeaşi locaţie în toate paginile memoriei de control iar vectorul de condiţii de test va activa pagina ce specifică microinstrucţiunea următoare care se va executa.

3. *memorie de control cu organizare pe blocuri*, Fig. 9.7.
   Pentru acest tip de organizare există două feluri de adrese:
   -adrese de microinstrucţiuni din acelasi bloc cu microinstrucţiunea curentă ;
   -adrese de blocuri .
   Împărţirea memoriei de microprograme în blocuri se face ţinând seama atât de structura microprogramului cât şi de resursele fizice disponibile. O organizare de acest fel conduce,

în general, la micşorarea lungimii microinstrucţiunii însă introduce timp suplimentar cu comutarea adreselor de blocuri.

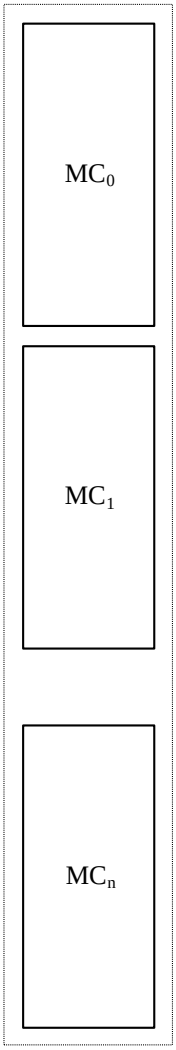


Figura 9.9
MC cu organizare pe blocuri

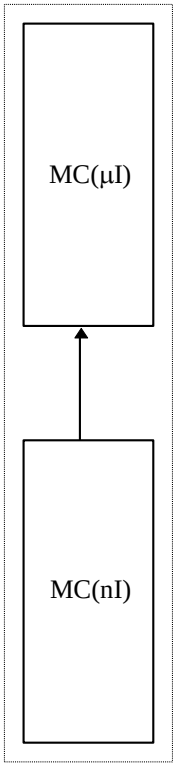


Figura 9.8
MC structurată pe două niveluri

4. *memorie de control divizată*, Fig. 9.8.

Memoria de microprograme divizată este alcătuită din două unităţi de memorie distincte :

MI - memorie de microinstrucţiuni, care păstrează toate microinstruciunile distincte posibile necesare pentru controlul resurselor fizice ;

MA - memorie de adrese de microinstrucţiuni, care păstrează programul specificat, nu prin microinstrucţiuni ce controlează resursele fizice, ci prin adrese de microinstrucţiuni (adrese pentru memoria MI).

În general numărul de tipuri de microinstrucţiuni distincte este mult mai redus decât microprogramul în sine, ceea ce implică ca numărul de biţi necesari pentru adresarea memoriei MI să fie şi el redus. In acest fel, lungimea cuvântului din memoria MA este

mult mai mic decât al memoriei MI, ceea ce conduce la o reducere substanţială a memoriei de control.

În schimb, pentru a executa o microinstrucţiune trebuie făcute două adresări, una la memoria MA şi una la memoria MI ceea ce conduce la un ciclu mai mare de microinstrucţiune.

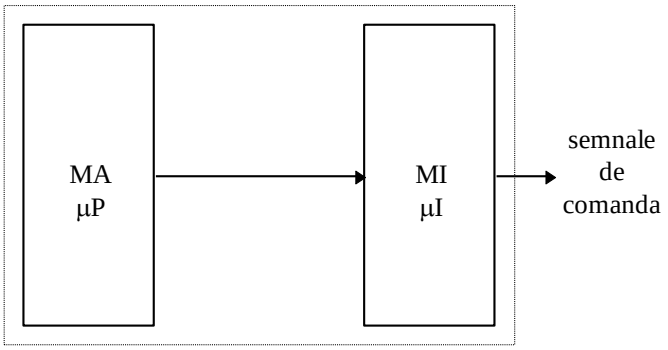


Figura 9.10
MC divizata

5. *memorie de control structurată pe două niveluri*, Fig. 9.9

La o astfel de organizare mecanismul citirii interpretării şi execuţiei unei instrucţiuni maşină este următorul :

- o instrucţiune maşină este interpretată de un set de microinstrucţiuni rezident în memoria de control MC (μI) ;
- la rândul ei, o microinstrucţiune este interpretată de o secvenţă de nanoinstrucţiuni rezidentă în memoria MC (nI) .

Această tehnică a nanoprogramării este conceptual echivalentă cu microprogramarea, structurarea pe două niveluri oferă o flexibilitate mai mare şi posibiltatea implementării unor structuri de control foarte complexe.

O astfel de organizare întâlnim la calculatorul NANODATA QM1.

9.1.2 Suportul fizic pentru păstrarea microprogramelor

Memoria de microprograme MC este o unitate de memorie de mare viteză care păstrează microprogramele ce se execută .

În ceea ce priveşte suportul fizic, pentru păstrarea microprogramelor, acesta poate fi realizat cu memorii PROM sau cu memorii RAM.

Realizarea memoriei de control cu componente de tip RAM conferă o caracteristică statică structurii microprogramate, în timp ce utilizarea componentelor de tip RAM oferă o caracteristică dinamică .

Caracteristica dinamică, permite utilizatorului ca printr-un ansamblu de mijloace software să aibă acces la microprogramul din memoria de control. Accesul poate consta în modificarea parţială, în completarea microprogramului pentru a crea noi primitive software sau chiar în modificarea totală a microprogramului, ceea ce înseamnă schimbarea arhitecturii maşinii de bază.

Este foarte important în a face deosebire între :

- maşini microprogramate, şi

- maşini microprogramabile

Prima categorie se referă la modalitatea de implementare a unităţii de comandă în sensul conceptului introdus de Wilkes fără a oferi resursele hardware şi suportul de programe pentru accesul utilizatorului la nivelul microprogramului.

Cea de a doua categorie oferă atât resursele hardware cât şi facilităţile software pentru accesul utilizatorului la nivelul microinstrucţiunilor.

Dintre calculatoarele cu memorie de control inscriptibilă amintim BURROUGH 1700, MICRODATA 1600, I102F, etc.

9.1.3 Organizarea logică a instrucţiunilor

Un cuvânt din memoria de control specifică un set de microoperaţii ce constituie componentele primitive ale controlului resurselor sistemului. Organizarea logică a microinstrucţiunilor este influenţată de :

- gradul de paralelism între microoperaţii, ce se doreşte realizat;
- structura maşinii de bază ;
- gradul de codificare sau de flexibilitate dorit ;
- gradul de optimizare al lungimii cuvântului de control .

Presupunând că microprogramele sunt specificate ca o secvenţă de seturi disjuncte de microoperaţii, se pot distinge mai multe modalităţi de codificare a acestora şi anume:

- codificare verticală sau maximală ;
- codificare orizontală sau cu control direct ;
- codificare minimală ;
- codificare cu control rezidual ;
- codificare cu control prin adrese ;
- codificare mixtă.

9.1.3.1 Codificare verticală

În cadrul codificării verticale, fiecare microinstrucţiune operaţională specifică o singură microoperaţie. Setul de microoperaţii (MO) necesar pentru controlul primitivelor funcţionale se codifică în [$\log_2$ | (MO)] biţi, ce constituie lungimea cuvântului din MC.

Pentru identificarea microoperaţiei specificate de µI se utilizează un decodificator, Fig. 9.9.

Această codificare reprezintă un caz extrem, deoarece elimină orice posibilitate de desfăşurare paralelă a operaţiilor elementare.

Din punctul de vedere al minimizării cuvântului de control, codificarea verticală implică numărul cel mai mic de biţi. Dimensiunea mare a decodificatorului face ca realizarea fizică a acestuia să aibe loc pe mai multe niveluri, ceea ce conduce la introducerea de întârzieri.

Un dezavantaj major al codificării maximale îl reprezintă eliminarea controlului paralel asupra resurselor precum şi inflexibilitatea dezvoltării sau completării sistemului în ceea ce priveşste introducerea de noi microoperaţii. Este aplicabilă numai în sisteme dedicate care au o structură specifică.

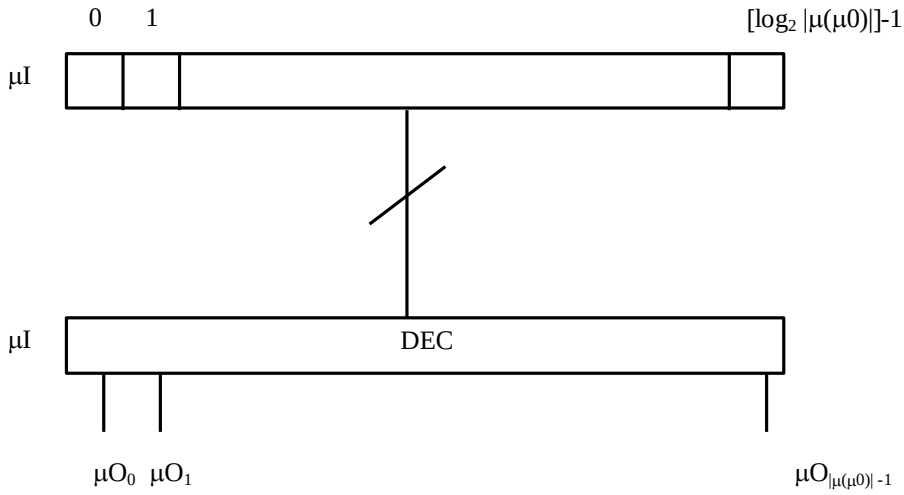


Figura 9.11
Codificarea verticală a microoperaţiilor

9.1.3.2 Codificarea orizontală

În cadrul acestei codificări, fiecare microoperaţie din setul (MO) este pusă în corespondenţă cu un bit din cadrul cuvântului de control. Controlul microoperaţiilor se face în mod direct, Fig. 9.11.

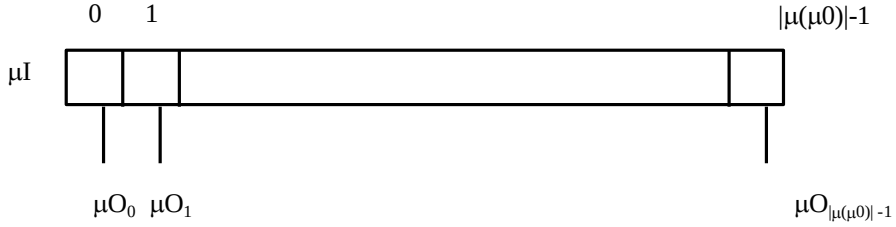


Figura 9.12
Codificarea orizontala

Codificarea orizontală realizează controlul tuturor microoperaţiilor paralele, posibile, ce se pot desfăşura în sistem.

Deşi oferă o flexibilitate mare şi asigură paralelismul maxim, utilizarea acestei codificări este un caz extrem datorită folosirii ineficiente a memoriei de control.

9.1.3.3 Codificarea minimală

Combină flexibilitatea şi paralelismul potenţial oferite de codificarea orizontală cu eficienţa codificării verticale.

Ideea de bază este de a grupa în clase de compatibilitate setul de microoperaţii care se exclud reciproc (μO dintr-o clasă de compatibilitate nu se vor efectua niciodată simultan).

Microinstrucţiunea este împărţită în câmpuri. Un câmp corespunde unei clase de compatibilitate. La nivel de câmpuri se realizează o codificare orizontală iar în cadrul câmpurilor se realizează o codificare verticală, Fig 9.12 a).

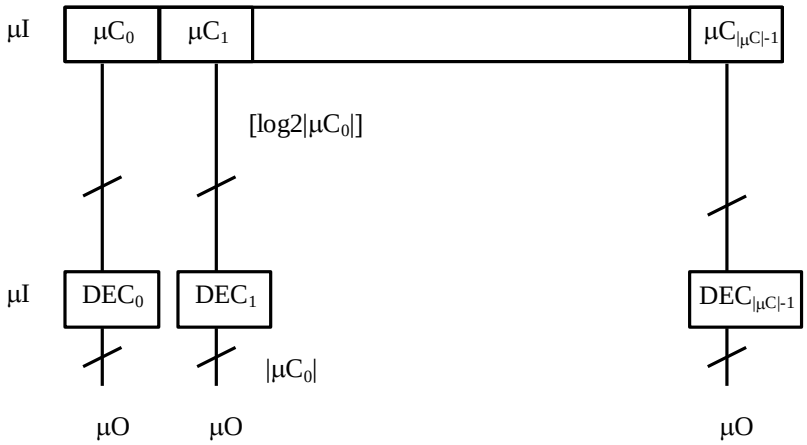


Figura 9.13 a)
Codificare minimală

Pentru un câmp μC_j care codifică $|\mu C_j|$ microoperaţii sunt necesari $|\log_2(|\mu C_j|+1)|$ biţi, deoarece trebuie să se prevadă şi posibilitatea de a nu specifica nici o microoperaţie din cadrul câmpului.

O variantă a acestei codificări o reprezintă codificarea pe două niveluri sau indirectă.

În codificarea pe două niveluri, validarea unor câmpuri depinde de valoarea altui câmp de control din microinstrucţiune, Fig 9.12 b).

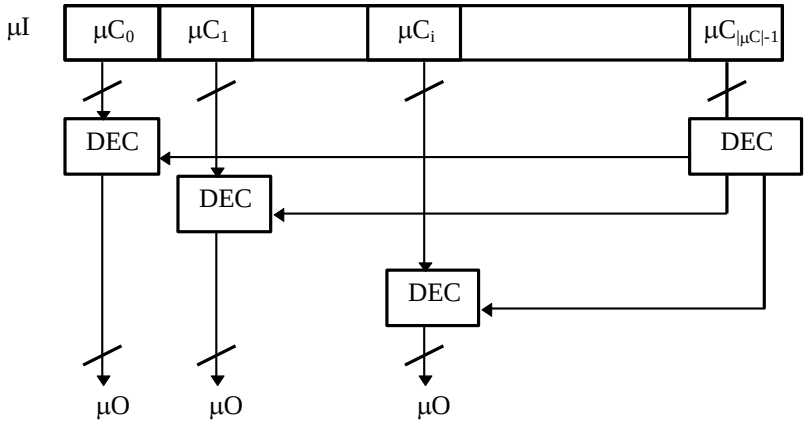


Fig. 9.12 b) Codificare minimală pe două niveluri

9.1.3.4 Codificarea cu control rezidual

Această metodă de codificare foloseşte pentru controlul primitivelor funcţionale registre de control rezidual. Cuvântul de control nu controlează direct resursele ci prin intermediul registrelor de control rezidual încărcate sub acţiunea microinstrucţiunilor, Fig. 9.13

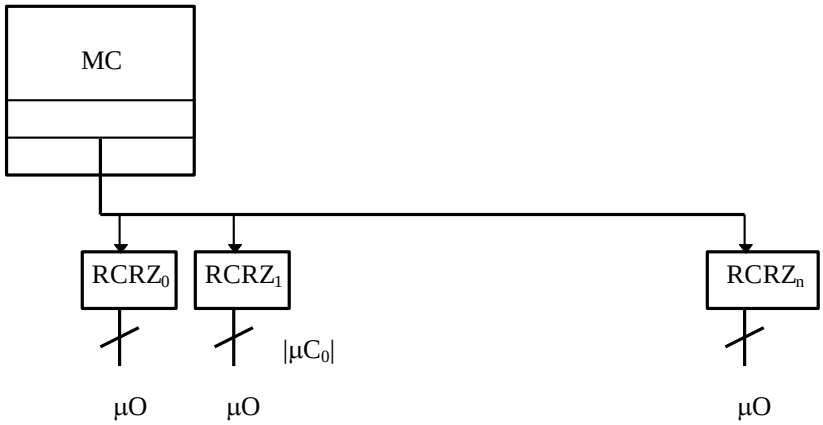


Figura 9.14
Codificare cu control rezidual

Microinstrucţiunile pot să înlocuiască sau să modifice valoarea unuia sau mai multor registre de control. Această tehnică, a controlului rezidual, asigură o economie de memorie de control atunci când unele primitive funcţionale realizează aceeaşi operaţie în mod repetat sau când un set de microoperaţii este activ o perioadă mare de timp, iar alte seturi de microoperaţii se modifică.

Registrele de control rezidual $RCRZ_j$, care specifică microoperaţiile de control al resurselor hardware pot fi manevrate cu ajutorul unor microinstrucţiuni de dimensiuni reduse.

9.1.3.5 Codificarea cu control prin adrese

O modalitate de implementare a microinstrucţiunilor operaţionale este aceea în care nu se specifică direct microoperaţiile care trebuie să se desfăşoare ci se specifică o adresă în cadrul unei memorii, unde sunt memorate toate microinstrucţiunile distincte posibile ce controlează sistemul, Fig. 9.14.

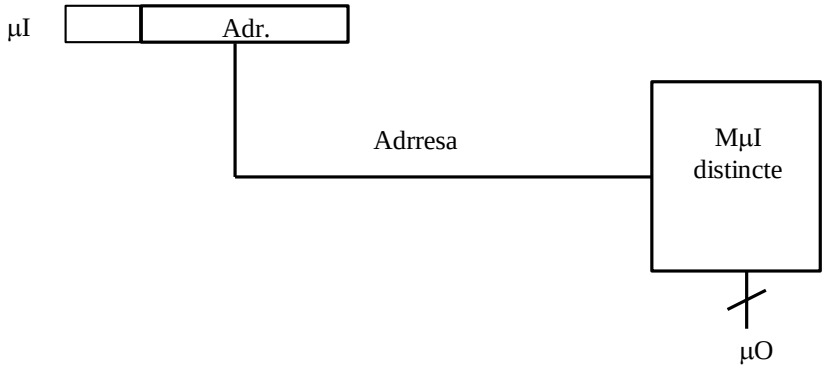


Figura 9.15
Codificare cu control prin adrese

Această modalitate reprezintă o formă simplificată a conceptului de nanoprogramare.

Numărul de microinstrucţiuni distincte nu depinde de numărul de resurse controlate ci de numărul de μO disticte, de mărimea μP şi de numărul de variabile de stare testate .

Memoria care păstrează microinstrucţiunile distincte va avea lungimea cuvântului suficient de mare pentru a controla toate microoperaţiile care se pot efectua simultan. Trebuie notat că fiecare μI este memorată o singură dată.

O astfel de implementare, face ca microprogramul să fie format dintr-o secvenţă de adrese care apelează μI păstrate în memoria de μI.

Un dezavantaj al acestei metode constă în faptul că necesită două accese la memorie în cadrul unui ciclu de microinstrucţiune, dar în schimb se realizează o economie importantă de memorie.

9.1.3.6 Codificare mixtă

O variantă utilizată mult în practică este aceea în care microinstrucţiunea este împărţită în câmpuri. Unele câmpuri controlează direct microoperaţiile (sub formă codificată sau directă) iar altele specifică adrese de memorie ce conţine un subset de microinstrucţiuni distincte, Fig. 9.15.

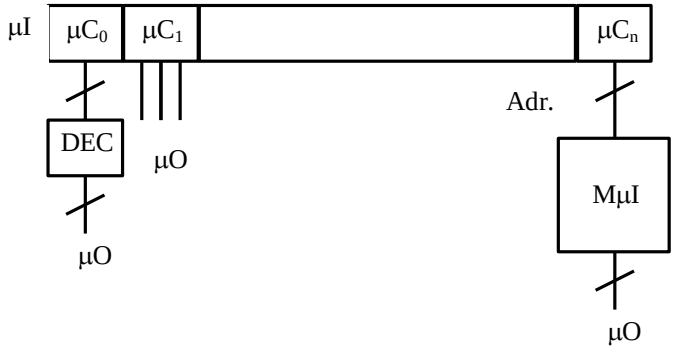


Figura 9.16
Codificare mixtă

9.1.4 Implementarea microinstrucţiunilor

Microinstrucţiunile sunt citite, interpretate şi executate de către microsecventiator (μS) în acelaşi fel în care o unitate de comandă convenţională execută instrucţiuni maşină.

Se poate defini o caracteristică serie-paralel care măsoară cantitatea de suprapunere între faza de execuţie a μI curente şi citirea, interpretarea μI următoare.

Din punctul de vedere al caracteristicii serie-paralel distingem trei tipuri de implementări :

- implementare serie;
- implementare serie-paralelă ;
- implementare paralelă.

Fie CI-faza de citire interpretare şi E-faza de execuţie a unei μI.

În implementarea serie, citirea microinstrucţiunii următoare nu începe decât după ce s-a terminat execuţia microinstrucţiunii curente, Fig. 9.16.

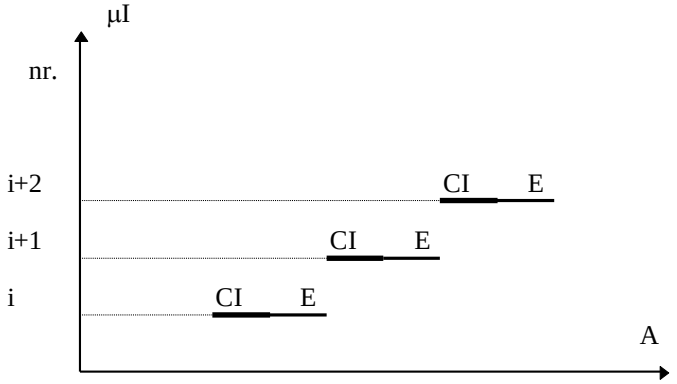


Figura 9.17
Implementare serie

În implementarea paralelă, Fig. 9.17, faza de citire a μI următoare se desfăşoară în acelaşi timp cu execuţia μI curente.

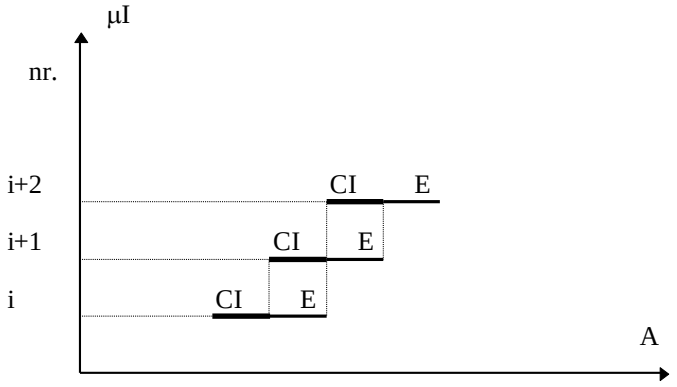


Figura 9.18
Implementare paralel

Se observă că ciclul de execuţie al microprogramului se reduce.

O astfel de implementare impune existenţa unor resurse fizice de comandă (microsecvenţiator, etc.) mai complexe, care să poată anticipa adresa microinstrucţiunii următoare chiar şi în cazul salturilor condiţionate.

O îmbinare a performanţelor implementării paralele şi a costului redus al implementării serie este realizată de implementarea serie-paralelă a microinstrucţiunilor, Fig. 9.18.

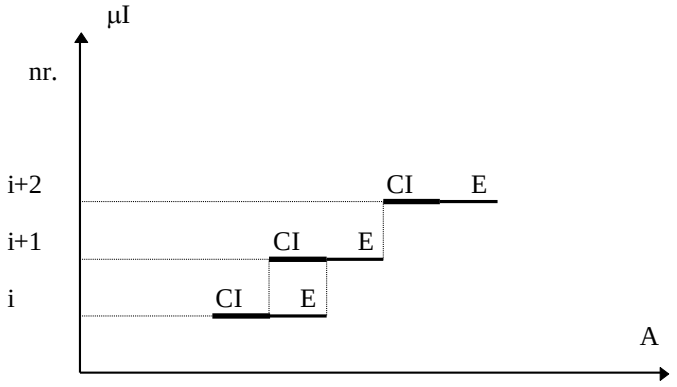


Figura 9.19
Implementare serie-paralel

Faza de citire a μI următoare se desfăşoară fie în timpul execuţiei μI curente, fie după terminarea ei, în funcţie de tipul microinstrucţiunii curente.

Succesiunea de microinstrucţini operaţionale se desfăşoară prin suprapunerea fazelor de citire şi execuţie, iar cele imediat următoare unor ramificaţii se citesc după terminarea execuţiei μI curente (de ramificaţie).

Implementarea serie-paralelă este caracteristică arhitecturilor "pipe line", şi este cea mai folosită.

Referitor la implementarea microinstrucţiunilor se poate defini şi o caracteristică monofază-polifază care se referă la numărul de faze utilizate într-un ciclu de microinstrucţiune.

Într-o implementare monofază, microoperaţiile sunt generate simultan, toate semnalele de control specificate de microinstrucţiune fiind active în acelaşi timp (pentru cele de tip impuls se ţine seama de front), Fig. 9.19.

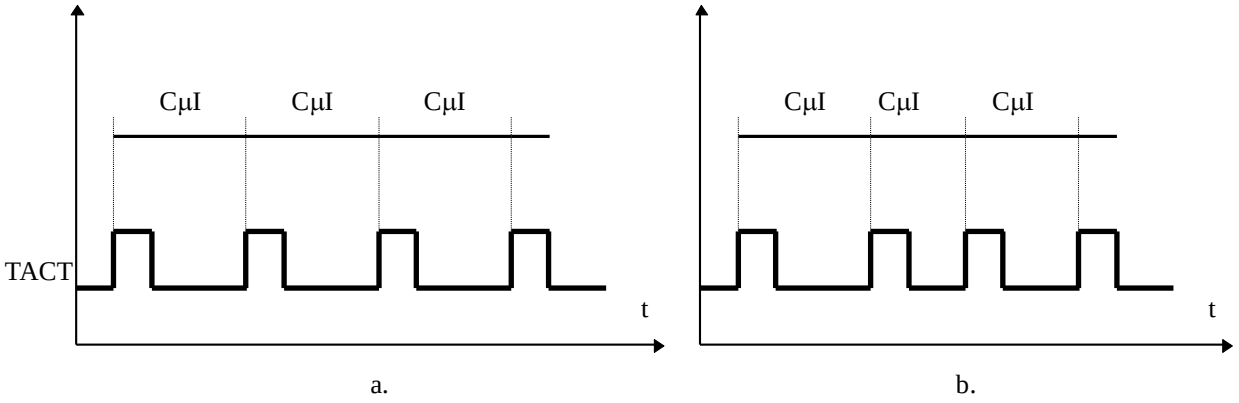


Figura 9.20
Implementare monofază
a. ciclul macroinstrucţiunii CμI constant
b. ciclul macroinstrucţiunii CμI variabil

Într-o implementare polifazică, microinstrucţiunea este activă o perioadă de tact însă semnalele de control sunt repartizate pe faze ale ciclului, Fig. 9.20.

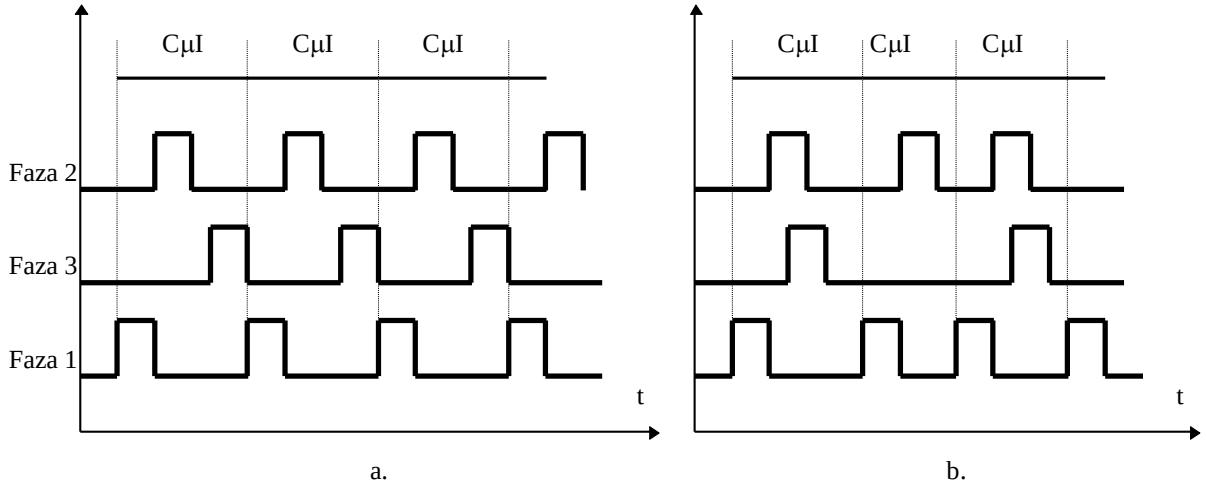


Figura 9.21
Implementare polifază
a. ciclul microinstructiunii constant
b. ciclul microinstructiunii variabil

Ciclul unei microinstrucţiuni poate fi constant sau variabil în sensul că poate avea aceeaşi durată pentru toate μI, respectiv o durată pentru un grup de microinstrucţiuni şi altă durată pentru un alt grup de μI.

9.2 **Proiectarea unităţii de comandă microprogramată** a calculatorului didactic

Proiectarea unităţii de comandă microprogramată a calculatorului didactic implică rezolvarea următoarelor aspecte:

- specificarea caracteristicilor unităţii de comandă microprogramate pentru calculatorul didactic ;
- specificarea detaliată a structurii calculatorului didactic;
- stabilirea operaţiilor elementare pentru controlul resurselor calculatorului didactic;
- stabilirea variabilelor de stare (condiţii de test);
- stabilirea formatului microinstrucţiunilor;
- proiectarea microsecvenţiatorului;
- descrierea microprogramului.

9.2.1 Specificarea caracteristicilor unităţii de comanda microprogramate pentru calculatorul didactic

Aspectele principale care trebuie considerate în vederea specificării unităţii de comandă microprogramată au fost prezentate în paragraful 9.1. Analizând aceste aspecte se constată că există o mare varietate de soluţii şi opţiuni pentru specificarea unităţii de comandă.

Având în vedere structura calculatorului didactic se poate proiecta unitatea sa de comandă sub forma microprogramată cu următoarele atribute generale:

- din punctul de vedere al relaţiei poziţionale între MC şi memoria principală se va alege o memorie de control separată de memoria principală;
- implementarea memoriei de control se va face cu memorii de tip PROM;
- memoria de control se va organiza asfel încât să se specifice o singură microinstrucţiune într-un cuvânt de control;
- organizarea logică a microinstrucţiunilor va fi de tip minimal (microinstrucţiunea împărţită în câmpuri);
- implementarea microinstrucţiunilor va fi de tip serie-paralelă;
- se va considera o implementare monofazü a microinstrucţiunilor.

Având în vedere caracteristicile prezentate, structura generală a unităţii de comandă microprogramată pentru calculatorul didactic se prezintă în Fig. 9.22.

unde:

MC - memoria de control, păstrează microprogramul care implementează fazele de citire interpretare şi execuţie ale instrucţiunilor maşină. Este de tip PROM. Un cuvânt de control din MC, echivalent cu o microinstrucţiune, generează toate semnalele de comandă (microoperaţiile) ce sunt active la un moment dat pe durata unei perioade de sincronizare.

μRI - registrul de microinstrucţiuni, păstrează microinstrucţiunea curentă ce se execută.

μCP - registrul de adrese al MC, păstrează adresa microinstrucţiunii următoare care se va citi.

μS - microsecvenäiatorul, reprezintă unitatea de comandă convenţionala care asigură citirea, interpretarea şi validarea acţiunilor microinstrucţiunilor. Pe baza tipului microinstrucţiunii curente, a stării primitivelor funcţionale ale calculatorului didactic precum şi a codului operaţie a instrucţiunii maşină curente (ce interpretează şi se execută), μS formează adresa μI următoare şi validează (activează spre resursele unităţii de execuţie) microoperaţiile din μI curentă.

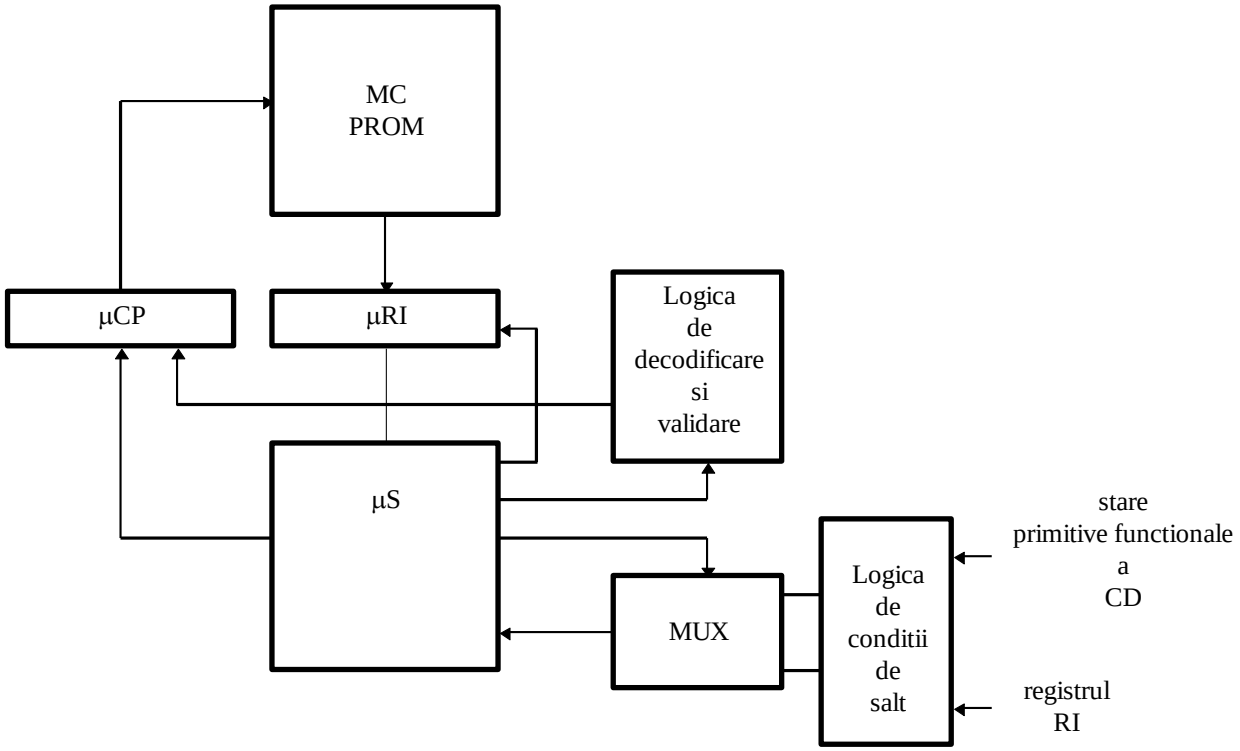
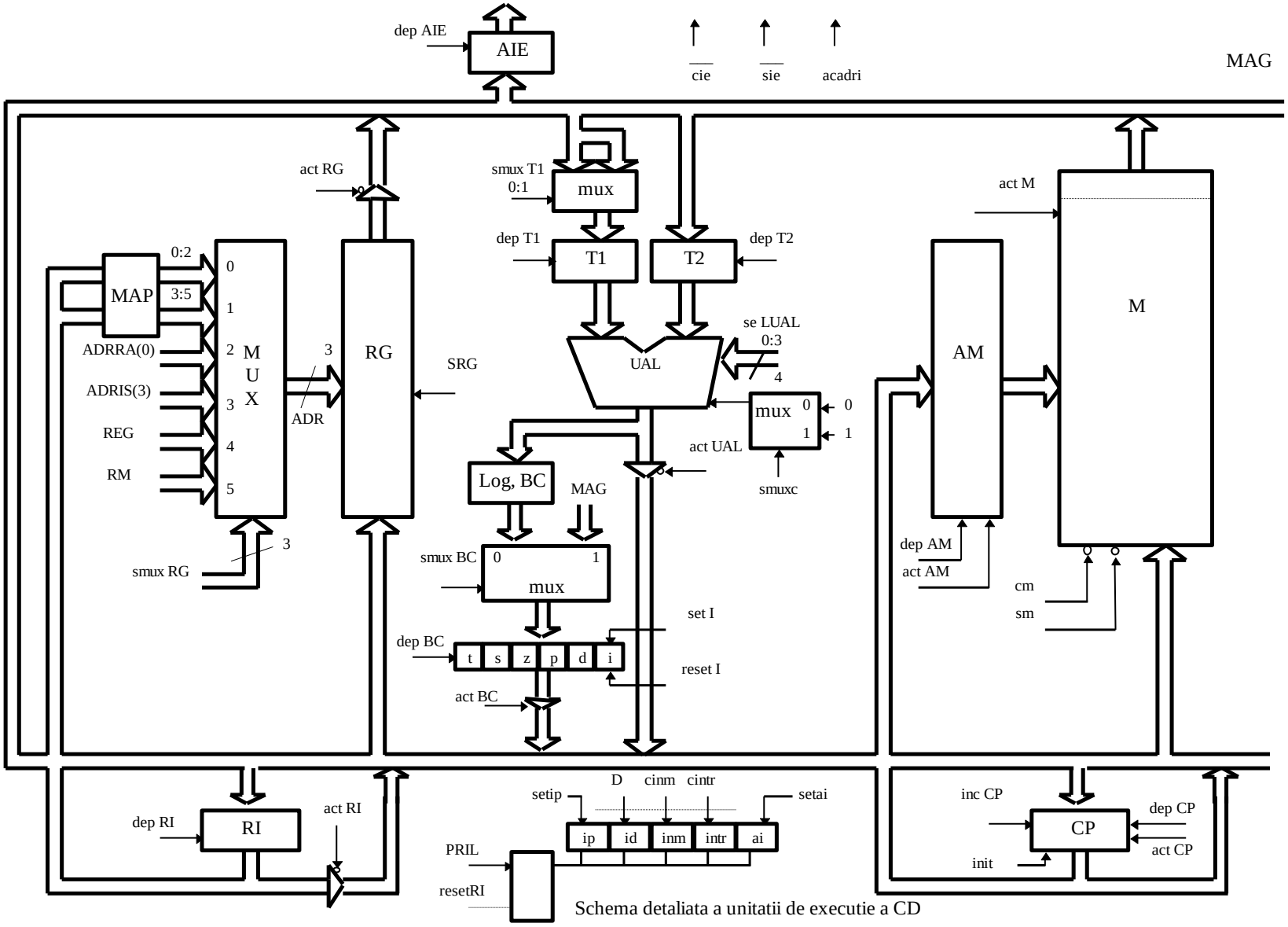


Figura 9.22

9.2.2 Specificarea detaliată a structurii calculatorului didactic

Pentru a stabili formatul microinstrucţiunilor şi a proiecta microsecvenţiatorul este necesar să detaliem structura părţii operaţionale (unităţii de execuţie) a calculatorului didactic.

În Fig.9.23 se prezintă structura detaliată a unităţii de execuţie a calculatorului didactic, fără a include subsistemul de intrări/ieşiri.



Schema detaliata a unitatii de executie a CD

Figura 9.23

Este necesar să precizam că toate resursele care au acces la magistrala MAG, trebuie să fie prevăzute cu ieşiri cu trei stări. Acest deziderat se poate realiza în două moduri :

- alegem resurse cu ieşiri cu trei stări (dacă există) ;
- alegem resurse care nu au ieşiri cu trei stări însă intercalăm între ieşirile lor şi magistrală circuite separatoare cu trei stări ("buffere tri state ").

În schema detaliată nu se precizează tipul componentelor cu care se face implementarea ci se stabilesc numai resursele şi operaţiile elementare asupra lor. Se pun în evidenţă funcţiile logice pe care trebuie să le efectueze primitivele funcţionale.

De exemplu, registrul CP indiferent de componenta cu care se implementează, trebuie să fie controlat de semnale (microoperaţii) care specifică :

- iniţializarea (ştergerea)- initl ;
- incrementarea - incCP ;
- încărcarea cu o valoare specificată de conţinutul magistralei MAG -depCP ;
- activarea pe magistrala MAG - actCP .

9.2.3 Stabilirea operaţiilor elementare pentru controlul resurselor calculatorului didactic

Analizând schema detaliată a unităţii de execuţie a calculatorului didactic, unde s-a precizat modul de utilizare a primitivelor funcţionale rezultă lista semnalelor de comandă:

1.actAM - activează registrul de adrese al memoriei pe liniile de adrese spre memorie (s-a considerat că trebuie să fie dezactivat când intervine DMA-ul);
2.depAM - depune (încarcă) conţinutul magistralei MAG în AM;
3.cm - comandă operaţia de citire din memorie;
4.sm - comandă operaţia de scriere în memorie;
5.actAM - activează conţinutul celulei de memorie adresate pe magistrala MAG;
6.incCP - incrementează CP ;
7.depCP - depune în CP conţinutul magistralei MAG ;
8.actCP - activează conţinutul registrului CP pe magistrala MAG ;
9.depRI - depune în RI conţinutul magistralei MAG ;
10.actRI - activează conţinutul registrului RI pe magistrala MAG;
11.smux$RG_{0:2}$ - selectează adresa memoriei locale RG, specificând astfel registrul general implicat în operaţia curentă ;
12.sRG - comandă scrierea în memoria locală RG ;
13.actRG - activează conţinutul memoriei locale RG pe magistrala MAG (conţinutul registrului general selectat de smuxRG) ;
14.smux$T1_{0:1}$ - selectează informaţia care se înscrie în registrul T1;
15.depT1 - depune date (încarcă) în registrul T1 ;
16.depT2 - depune date (încarcă) în registrul T2 ;
17.sel$UAL_{0:3}$ - specifică operaţia ce trebuie efectuată de UAL ;
18.actUAL - activează rezultatul din UAL pe magistrala MAG ;
19.smuxC - selectează informaţia pentru transportul de intrare în UAL
20.smuxIND - selectează informaţia ce se înscrie în indicatorii de condiţii ;
21.depIND - depune (încarcă) datele selectate de multiplexorul smuxIND în indicatorii de condiţii IND (t,s,z,p,d,I) ;
22.actIND - activează pe magistrala MAG conţinutul indicatorilor de condiţii IND ;
23.depAIE -depune (încarcă) în registrul AIE informaţia ce se găseşte pe magistrala MAG (adresa portului de intrare/ieşire implicat în transfer);

24.cie - activează pe magistrala MAG conţinutul portului de intrare (citeşte port de intrare) ;
25.sie - comandă scrierea într-un port de ieşire ;
26.setI - înscrie "1" (setează) în indicatorul I ;
27.resetI - înscrie "0" (resetează) în indicatorul I;
28.setip - înscrie "1" (setează) în bistabilul ip de cerere de întrerupere prin program ;
29.setai- înscrie "1" în bistabilul de achitare întrerupere ai
30.resetbi - înscrie "0" (resetează) în bistabilul de cerere de întrerupere corespunzător cererii luate în considerare şi în bistabilul de achitare întrerupere (ai) ;
31.actADRI - activează pe magisrala MAG adresa celulei capcană corespunzătoare nivelului de întrerupere ce urmează să fie tratat ;

9.2.4 Stabilirea variabilelor de stare (condiţiilor de test)

Pentru a asigura ramificaţiile în cadrul microprogramului, microsecvenţiatorul (μS) testează starea primitivelor funcţionale (caracterizată prin valoarea indicatorilor de condiţii) şi biţi din registrul de instrucţiuni (cod operaţie, mod de adresare).

Lista variabilelor de stare (semnalelor testate) în fazele de citire interpretare şi execuţie a instrucţiunilor maşină rezultă din analiza programului AHPL descris în capitolul 5 şi este pezentată în cele ce urmează .

În lista semnalelor testate se precizează semnalul sau combinaţia de semnale analizate precum şi pasul AHPL în care are loc testul.

Lista semnalelor testate (sau a funcţiilor logice testate) în programul ce descrie fazele de citire interpretare şi execuţie a instrucţiunilor maşină a calculatorului didactic se prezintă in tabelul 9.1.

În lista semnalelor testate s-au pus în clar toate condiţiile care au apărut în programul AHPL. Această listă se poate reduce din mai multe puncte de vedere şi anume:

- să se specifice numai semnalele directe iar decizia de testare pe "0" logic sau pe "1" logic să fie luată prin microprogram;
- grupurile de semnale specificate prin funcţii logice pot fi testate individual, însă în mai multe microinstrucţiuni de salt ceea ce conduce la scăderea performantelor unităţii de comandă microprogramată.

Tabelul 9.1

Nume semnal sau funcţie logică testată	pas AHPL
$\overline{\text{1.SL(SYN(starta)}}$	1
2.id v inm	2
$\overline{\text{3.inm} \vee \text{id} \wedge (\text{intr} \wedge \text{I})}$	2
$4.RI_0$	4
$5.RI_8 \wedge RI_9$	5,37,39,54
$\overline{6.RI_8} \wedge RI_9 \wedge RI_{13} \wedge RI_{14}$	5
$\overline{7.RI_9} \wedge RI_{13}$	7
$8.RI_9 \wedge RI_{13}$	7
$9.RI_9$	10
$\overline{10.RI_8}$	11
$\overline{11.RI_{15}}$	22
$12.RI_1$	26
$13.RI_3$	27,60,66
$14.RI_4$	28
$15.RI_5$	28
$16.RI_2$	29,57,60
$\overline{17.RI_7}$	30,59
$\overline{18.RI_7 \wedge RI_8 \wedge RI_9}$	30,59
$19.RI_7 \wedge RI_8 \wedge RI_9$	30,59
$20.RI_7$	31,58
$\overline{21.RI_7} \wedge \overline{RI_8 \wedge RI_9}$	31,58
$\overline{22.RI_7} \wedge RI_8 \wedge RI_9$	31,58
$\overline{23.RI_8 \wedge RI_9}$	37,39,54
$24.RI_6$	38,53,69,72
$25.RI_{4:6}$	55,60
$\overline{26.RI_2}$	60
$\overline{27.RI_4} \wedge RI_5$	67
$\overline{28.RI_4} \wedge RI_5$	67
$29.RI_4 \wedge RI_5$	67
$\overline{30.RI_6}$	81
31.indicatori condiţii + funcţii de indicatori condiţii	85
32.salt necondiţionat "1"	15,16,18,32,37,42,47,53,56,65,70,71,75,78,83,88

9.2.5 Stabilirea formatului microinstrucţiunilor

Pentru unitatea de comandă microprogramată a calculatorului didactic vom defini două tipuri de microinstrucţiuni şi anume:

- microinstrucţiune operaţionala, care specifică controlul primitivelor funcţionale ale unităţii de execuţie ;
- microinstrucţiune de ramificaţie, care permite testarea stării primitivelor funcţionale şi asigură ramificaţia în microprogram .

Pentru a putea stabili structura microinstrucţiunilor trebuie analizate următoarele aspecte:

- lista semnalelor care controlează primitivele funcţionale ale unităţii de execuţie ;
- programul AHPL care descrie secvenţa de comandă a fazelor de citire interpretare şi execuţie a instrucţiunilor maşină pentru a stabili setul de microoperaţii elementare ce trebuie să se desfăşoare în paralel ;
- lista semnalelor (funcţiilor logice de semnale) care reprezintă condiţiile de test în cadrul algoritmului ce se implementează în microprogram .

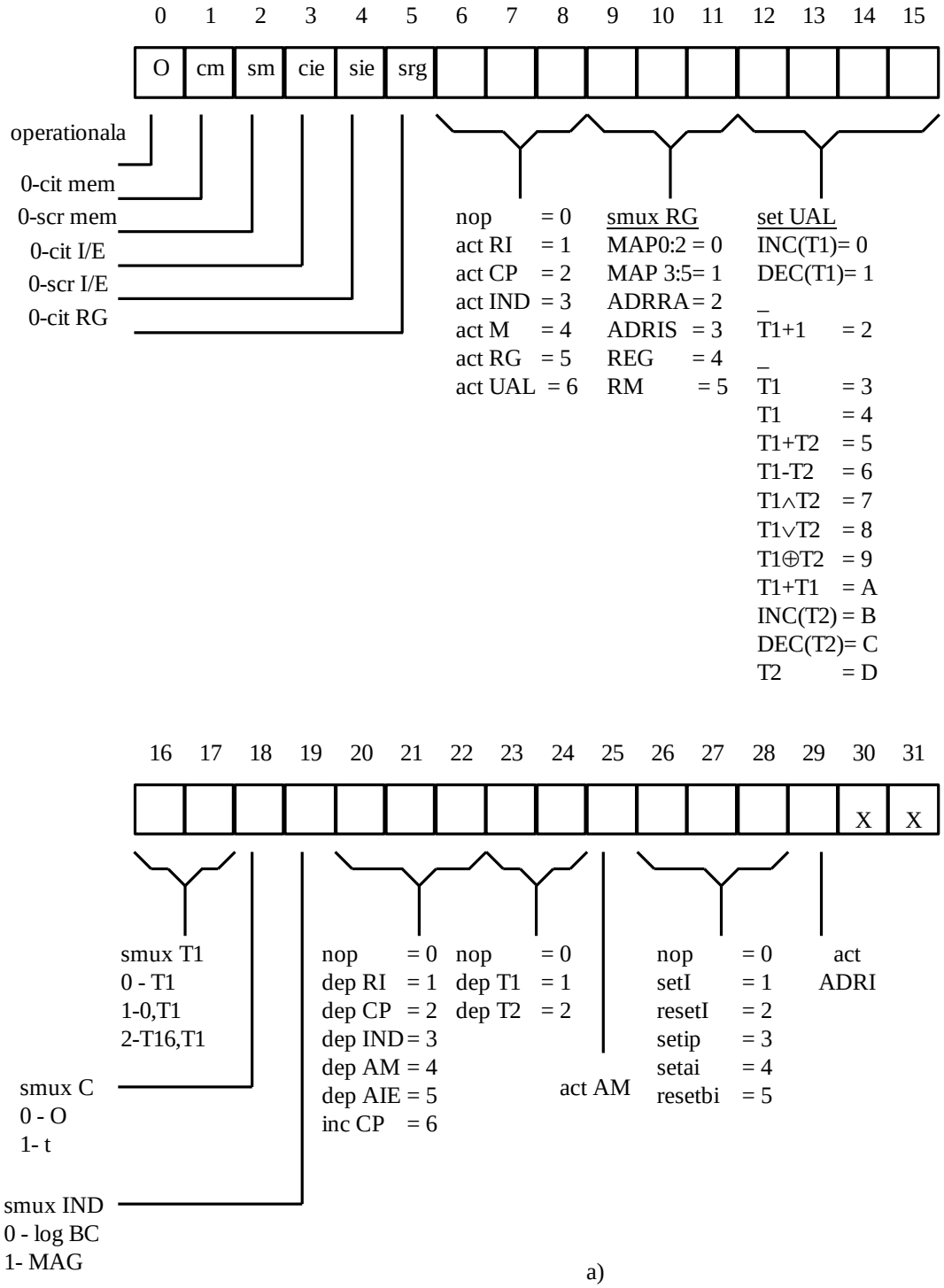
Listele de semnale care controlează primitivele funcţionale respectiv condiţiile de test au fost prezentate anterior.

Din analiza programului AHPL care descrie fazele de citire interpretare şi execuţie a instrucţiunilor maşină pentru calculatorul didactic (prezentat în capitolul 5) rezultă seturile de operaţii elementare ce trebuie să se desfăşoare în paralel. În cele ce urmează se prezintă seturile de operaţii elementare ce se desfăşoară în paralel şi paşii AHPL când sunt ele active (lista este alcătuită numai pentru faza de citire interpretare pas 1:25).

Lista seturilor de microoperaţii elementare paralele :

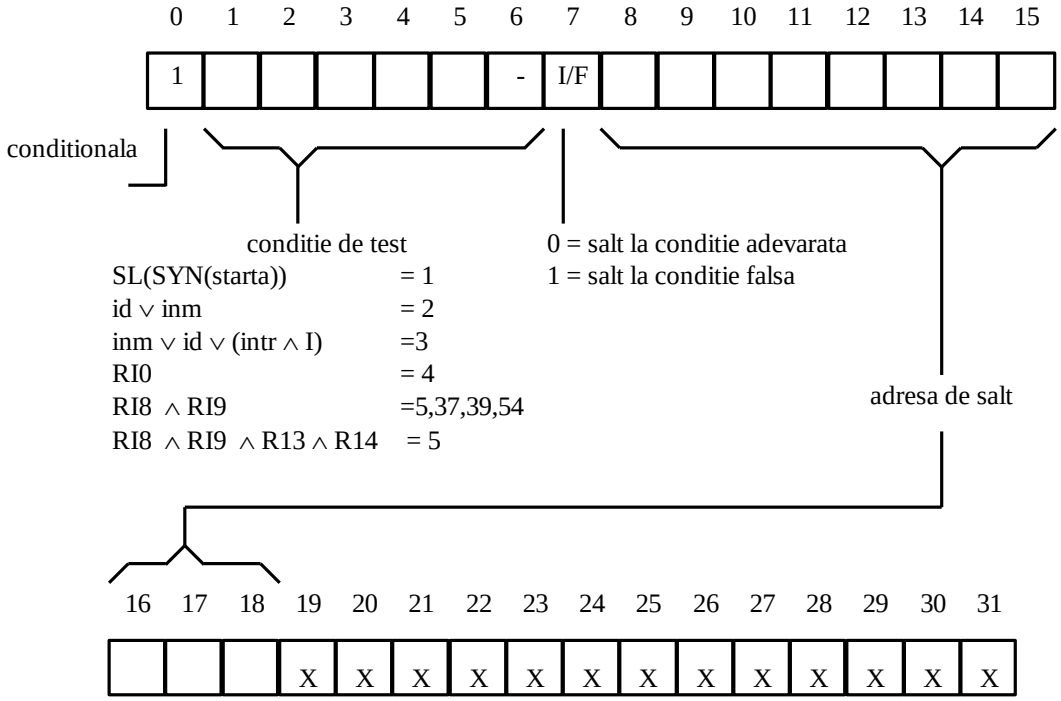
Nr. crt.	Seturi de microoperaţii paralele	pas AHPL
1.	setai	2.1
2.	actCP;depAM	2.2;13;20
3.	actAM;cm;actM;depRI	3
4.	smuxRG=0;actRG;smuxT1=0;depT1	6
5.	smuxRG=1;actRG;depT2	8
6.	selUAL=5;actUAL;smuxT1=0;depT1	9;15
7.	incCP	12;19
8.	actAM;cm;actM;depT2	14
9.	selUAL=0BH;actUAL;smuxRG=1;sRG	16
10.	smuxRG=1;actRG;depT2	17
11.	selUAL=0CH;actUAL;smuxRG=1;sRG;depT2	18
12.	actAM;cm;actM;smuxT1=0;depT1	21;24
13.	selUAL=4;actUAL;depAM	23;25
14.	etc pentru paşii	25...

Formatul microinstrucţiunilor sunt prezentate în Fig 9.24, cele operaţionale Fig.9.24 a) iar cele de ramificaţie în Fig.9.24 b).



a)

Figura 9.24 a)
Structura microinstructiunilor operaţionale



b)

Fig 9.2 b) Structura microinstrucţiunilor condiţionale

În proiectarea microinstrucţiunilor s-a ţinut seama că microoperaţiile paralele prezentate anterior să facă parte din câmpuri distincte şi de faptul că anumite microoperaţii se exclud reciproc (cum ar fi activarea unor resurse pe magistrala MAG).

De asemenea s-a considerat, pentru a mări flexibilitatea în elaborarea microprogramelor, că decizia în funcţie de testarea unei condiţii să poată fi luată la valoare adevărată sau falsă. Pentru aceasta s-a prevăzut bitul 7 în microinstrucţiunea de ramificaţie care face un "sau exclusiv " cu condiţia de test.

Lungimea microprogramului s-a considerat de maximum 2048 de locaţii, rezultând astfel un număr de 11 biţi în formatul microinstrucţiunii de ramificaţie pentru adresa de salt (biţii 8..18).

Observaţie: Având în vedere faptul că în general memoriile de tip PROM sunt organizate pe 4 sau 8 biţi, desi lungimea exactă a microinstrucţiunii este de 30 de biţi, s-a preferat să se lucreze cu o lungime de 32 de biţi.

9.2.6 Proiectarea microsecvenţiatorului

Având în vedere structura microinstrucţiunilor se poate descrie funcţionarea microsecvenţiatorului (μS) care asigură citirea interpretarea şi execuţia microinstrucţiunilor (activarea microoperaţiilor spre unitatea de execuţie a CD).

Programul AHPL de descriere a unităţii de comandă microprogramată a calculatoului didactic este :

MODULE: Unitatea de comandă microprogramată

MEMORY: MC [2048;32]; mRI [32]; mCP [11]

INPUTS: lista condiţiilor de test

OUTPUTS: lista semnalelor de comandă

1. µRI <- **BUSFN**(MC;DCD(µCP))

/* se citeşte microinstrucţiunea curentă în registrul de microinstrucţiuni µRI */

2. -> μRI_0 /(4)

/* se separă microinstrucţiunile de ramificaţie la pas 4 şi se continuă cu cele operaţionale */

3. cm,sm,cie,sie,sRG = $\mu RI_{1:5}$;

actRI,actCP,actIND,actM,actRG,actUAL = $DCD_{1:6}(\mu RI_{6:8})$;

$smuxRG_{0:2} = \mu RI_{9:11}$;

$selUAL_{0:3} = \mu RI_{12:15}$;

$smuxT1_{0:1} = \mu RI_{16:17}$;

smuxC = μRI_{18} ;

smuxIND = μRI_{19} ;

depRI,depCP,depIND,depAM,depAIE,incCP = $DCD_{1:6}(\mu RI_{20:22})$;

depT1,depT2 = $DCD_{1:2}(\mu RI_{23:24})$;

actAM = μRI_{25} ;

setI,resetI,setip,setai,resetbi =DCD1:5($\mu RI_{26:28}$) ;

actADRI = μRI_{29} ;

µCP <- INC(µCP) ;

-> (1)

/* se activează câmpurile de microoperaţii spre unitatea de execuţie, se incrementează contorul de adrese microinstrucţiuni şi se revine la pas 1 pentru citire microinstrucţiunii următoare

4. µCP - ($\overline{INC}$(µCP) ! $\mu RI_{8:18}$) * (g,g)

-> (1)

/* se formează adresa microinstrucţiunii următoare care poate fi adresa curentă incrementată (dacă condiţia de salt nu este îndeplinită) sau adresa de salt din microinstrucţiune (dacă condiţia de salt este îndeplinită) după care se merge la citirea microinstrucţiunii următoare */

END SEQUENCE

S-a considerat că funcţia g reprezintă condiţia de test şi are următoarea formă :

$g = \mu RI_7 \oplus$ (SL(SYN(starta))! id v inm! inm v id (intr^I)! RI_0! RI_8^RI_9! .....RI_6! COND! 1)*$DCD_{0:31}(\mu RI_{1:6})$

Semnalul de test COND reprezintă ieşirea unui multiplexor care are ca intrări indicatorii de condiţii sau funcţii logice dintre aceştia conform descrierii din tabelul 5.5, iar ca selecţie biţii $RI_{4:7}$ din codul operaţie al instrucţiunii de salt condiţionat.

9.2.6.1 Descrierea microprogramului

Descrierea microprogramului care implementează unitatea de comandă microprogramată a calculatorului didactic se poate face utilizând un limbaj de microasamblare. Pentru aprofundarea relaţiei între cuvântul de control şi semnalele de comandă ce controlează primitivele funcţionale ale unităţii de execuţie se exemplifică descrierea unei secţiuni de microprogram specificând direct biţii din microinstrucţiune.

1. -> $\overline{SL(SYN(starta))}$ /(1)

2. -> (inm v id, inm v id v (intr ^ I)) / (91,2.2)

2.1 ai <- 1 ;

-> (91)

2.2 AM <- CP

Trebuie remarcat faptul că stabilindu-se microinstrucţiuni de tip operaţional şi de tip condiţional, paşii AHPL care specifică acţiune şi salt vor fi implementaţi în două microinstrucţiuni distincte iar paşii AHPL care specifică salturi multiple se implementează prin mai multe microinstrucţiuni se ramificaţie.

Imaginea memoriei de control pentru secvenţa prezentată anterior este următoarea:

```
pas 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 15 16 17 18 19 20 21 22
23 24 25 26 27 28 29 30 31

1.  1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 x x
2.  1 0 0 0 0 0 0 1 0 ..adresa pasului 91 ... x x x x x x x x x x x x x x
2.  1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 1 x x x x x x x x x x x x x x
2.1 0 1 1 1 1 1 1 0 0 0 x x x x x x x x x x x x x 0 0 0 0 0 0 1 0 0 0 x x
2.1 1 0 1 1 1 1 1 1 0 ..adresa pasului 91 ... x x x x x x x x x x x x x x
2.2 0 1 1 1 1 1 1 0 1 0 x x x x x x x x x x x x x 1 0 0 0 0 0 0 0 0 0 x x
```

9.3 Codificarea minimă a unui set de microinstrucţiuni

În proiectarea unităţilor de comandă microprogramate, după ce s-a determinat setul de microinstrucţiuni complete ce realizează execuţia unui microsubbloc în minimum de paşi prin controlul tuturor operaţiilor paralele ce se pot executa în sistem, se pune problema codificării minime a acestui set de microinstrucţiuni.

Problema optimizării numărului de biţi constă în a stabili lungimea cuvântului memoriei de control ce specifică microinstrucţiunile, astfel încât aceasta să fie minimă, având în vedere asigurarea controlului tuturor microoperaţiilor paralele specificate de secvenţa de microinstrucţiuni ce descrie unitatea de comandă.

Practic se va stabili organizarea logică a microinstrucţiunilor prin specificarea câmpurilor şi a microoperaţiilor din fiecare câmp, astfel încât orice microinstrucţiune completă să poată fi specificată.

Stabilirea structurii logice a microinstrucţiunii va fi realizată având în vedere următoarele aspecte:

- microoperaţiile specificate de fiecare microinstrucţiune completă ;
- natura componentelor fizice cu care se va face implementarea memoriei de control.

Schwartz [SCHW] a fost primul care a formulat această problemă prin introducerea unui model de w cuvinte a N biţi fiecare, în care fiecare cuvânt putea specifica una sau mai multe microoperaţii. Algoritmul propus de el, bazat pe o enumerare exhaustivă, generează o codificare a setului de microoperaţii distincte într-un număr minim de câmpuri.

Grasselli şi Montanari [GRA] au arătat că soluţia cu un număr minim de câmpuri nu asigură număr minim de biţi. Ei au reformulat problema încadrând-o în teoria comutaţiei, reducând problema optimizării numărului de biţi la problema acoperirii mintermenilor cu implicanţi primi.

Problema este reluată de Das şi Barrejee [DAS] plecând de la clasele maximale de compatibilitate, generând solutii mai bune decât cele obţinute de Grasselli.

Toate aceste metode sunt bazate pe presupunerea că toate microoperaţiile specificate de microinstrucţiunile complete ale partiţiei microblocului, care se execută în paralel, se desfăşoară în cadrul aceleiaşi faze a ciclului memoriei de control. Sunt aplicabile numai pentru microinstrucţiunile monofazice.

Problema minimizării în cadrul microinstrucţiunilor polifazice a fost tratată de Dasgupta. În prezentarea pe care o face, a considerat ca microoperaţiile din acelasi câmp sunt executate în aceeaşi fază a ciclului microinstrucţiunii.

Problema generală a optimizării numărului de biţi este o problemă din clasa "NP complete". Apartenenţă la clasa "NP complete" a fost demonstrată de Robertson [ROBE79]. Având în vedere faptul că memoriile PROM sunt configurate în general pe 4 sau 8 biţi şi că problema generală a optimizării face parte din clasa "NP complete" este bine ca soluţiile adoptate pentru codificarea minimă a unui set de microinstrucţiuni să aibă şi un caracter ingineresc. în cele ce urmează o să se prezinte o modalitate de codificare aproape minimă a unui set de microinstrucţiuni bazată pe un algoritm cu caracter ingineresc.

Def. 1

Fie $mPt=\{mIC_1,mIC_2,...,mIC_{[mPt]}\}$ partiţia unui microsubbloc în microinstrucţiuni complete şi $MB(mO)=\{mO_1,mO_2,..., mO_{[MB]}\}$ setul de microoperaţii distincte din cadrul microsubblocului.

Două microoperaţii mO_i şi mO_j sunt **compatibile** dacă pentru orice k, $1 \le k \le [mPT]$, dacă $mO_i \in mIC_k$ atunci $mO_j \notin mIC_k$.

Compatibilitatea între două microoperaţii trebuie privită în sensul că cele două microoperaţii nu sunt specificate (nu sunt active) niciodată împreună în cadrul unei microinstrucţiuni din microbloc.

Controlul resurselor sistemului microprogramat nu necesită niciodată efectuarea în paralel a celor două microoperaţii.

Def. 2

Două microoperaţii $mO_i \in MB(mO)$, $mO_j \in MB(mO)$ sunt **incompatibile** dacă există cel puţin o microinstrucţiune completă mIC astfel încât $mO_i \in IC_k$ şi $mO_j \in mIC_k$.

Def. 3

O clasă de compatibilitate CC(mO) este un set (subset) al mulţimii MB(mO) în care oricare două microoperaţii sunt compatibile între ele.

$CC(mO)=\{mO \mid$ pt orice $mO_i, mO_j \in CC(mO)$ avem mO_i compatibilă cu $mO_j\}$

Def. 4

O clasă de compatibilitate maximă MCC(mO) este acea clasa de compatibilitate la care nu mai poate fi adăugată nici o microinstrucţiune fără a se pierde compatibilitatea.

$MCC(mO)=\{mO_j \mid$ pt orice $mO_i \notin MCC(mO)$, există $mO_j \in MCC(mO)$
astfel încât $mO_i \in mIC_k$ şi $mO_j \in mIC_k\}$.

În mod analog se defineşte clasa de incompatibilitate maximă MIC(mO).

Def. 5

Costul de implementare a unei clase de compatibilitate (măsurat în numărul de biţi necesari pentru codificare) este dat de implementarea codificării verticale a microoperaţiilor ce compun clasa.

Cost $CC_i=[\log_2(|CC_i|+1)]$

iar costul total de implementare al cuvântului de control

$$\text{Cost } CC = \sum_{i=1}^{k} [\log_2(|CC_i|+1)]$$

unde k este numărul de clase de compatibilitate.

Obs

O clasă de compatibilitate corespunde unui câmp din cadrul microinstrucţiunii.

Implementarea microinstrucţiunilor complete se face prin control rezidual pentru a putea specifica toate microoperaţiile paralele necesare.

Setul de clase compatibile $CC=\{CC_1,CC_2,...,CC_k\}$ poate specifica orice microoperaţie din multimea MB(mO) şi orice microinstrucţiune completă din cadrul partiţiei microsubblocului.

Noţiunea de compatibilitate poate fi extinsă în cadrul structurilor microprogramate cu ciclu polifazic, în sensul ca în cadrul aceleiaşi clase pot fi specificate microoperaţii ce se desfăşoară paralel, dacă ele se execută în faze diferite.

Se va descrie o metodă de codificare minimă a setului de microinstrucţiuni complete obţinut prin partiţionarea unui microsubbloc, plecând de la un subset de clase de compatibilitate maxime. Mai întâi vom face câteva precizări privind estimarea costului minim.

Estimarea costului minim

Problema codificării minime presupune două faze distincte:

- enumerarea tuturor claselor de compatibilitate maximă MCC;
- determinarea unui subset de MCC care să implementeze costul minim pentru codificare.

Costul implementării depinde de numărul de MCC necesar pentru acoperirea întregului set de microoperaţii şi de numărul de microoperaţii din fiecare clasă de compatibilitate maximă. De notat faptul că un cuvânt din memoria de control, adică o microinstrucţiune completă, este un set de clase incompatibile între microoperaţiile componente.

O clasă de compatibilitate specifică cel mult o microoperaţie din cadrul unui cuvânt al memoriei de control.

Def. 6

Pentru orice clasă de incompatibilitate IC, clasele de compatibilitate maximă ce conţin un element din IC se numesc clase de compatibilitate maximă asociate (AMCC) clasei de incompatibilitate.

Prop. 1

Pentru orice clasă de incompatibilitate maximă MIC reuniunea claselor de compatibilitate maximă asociate acoperă întreg setul de microoperaţii.

Dem.

Fie $MIC(mO)=\{mO_1,...,mO_{|MIC|}\}$ formată din |MIC| microoperaţii, unde $|MIC| \leq |MB|$. Presupunem contrariul şi anume că reuniunea claselor de compatibilitate maximă asociate nu acoperă întreg setul de microoperaţii. Fie mO_j una din aceste microoperaţii presupuse neacoperite cu proprietatea că:

$$mO_j \in MB(mO) \text{ dar } mO_j \notin MIC(mO) \text{ şi } mO_j \notin \bigcup_{MIC} AMCC.$$

Datorită faptului că nu aparţine reuniunii claselor de compatibilitate maximă asociate, rezultă faptul ca mO_j este incompatibilă cu toate microoperaţiile din MIC. Înseamnă că poate fi adăugată acestei clase, ceea ce contrazice definiţia clasei de incompatibilitate maximă. Ajungându-se la contradicţie rezultă că presupunerea este falsă.

Prop. 2

Pentru o clasă de incompatibilitate maximă MIC, reuniunea oricăror k clase de compatibilitate, $k < |MIC|$, nu poate acoperi setul de microoperaţii MB(mO).

Dem.

Considerăm $MIC(mO)=\{mO_1,mO_2,...,mO_{|MIC|}\}$. |MIC| este limita inferioară pentru numărul de clase de compatibilitate ce satisface acoperirea, deoarece o clasă de compatibilitate poate acoperi un singur element din MIC. Rezultă că orice reuniune de k clase de compatibilitate $k < |MIC|$ nu poate acoperi setul de microoperaţii.

Exemplul 1

Fie $MB(mO)=\{mO_1, mO_2, mO_3, mO_4, mO_5, mO_6, mO_7\}$ şi clasele de incompatibilitate maximale definite de structura microprogramată:

$MIC_1(mO)=\{mO_1,mO_2,mO_3,mO_4\}$

$MIC_2(mO)=\{mO_3,mO_4,mO_6,mO_7\}$

$MIC_3(mO)=\{mO_5,mO_6,mO_7\}$
$MIC_4(mO)=\{mO_1,mO_5\}$

Clasele de compatibilitate maximale sunt :
$MCC_1=\{mO_1,mO_6\}$ $MCC_2=\{mO_1,mO_7\}$ $MCC_3=\{mO_2,mO_5\}$
$MCC_4=\{mO_2,mO_6\}$ $MCC_5=\{mO_2,mO_7\}$ $MCC_6=\{mO_3,mO_5\}$
$MCC_7=\{mO_4,mO_5\}$ $MCC_8=\{mO_4,mO_6\}$ $MCC_9=\{mO_4,mO_7\}$

Clasele de incompatibilitate maximale sunt determinate pe baza grafului de dependenţă şi a conflictului de resurse şi reprezintă microoperaţiile specificate de microinstrucţiunile complete ce descriu microblocul.

Clasele MCC sunt determinate din matricea de microoperaţii ce indică incompatibilitatea. Aplicarea operatorului SAU între liniile matricii va specifica microoperaţiile ce fac parte din clasa de compatibilitate maximă.

Calcularea MCC este o binecunoscută problemă în teoria automatelor finite, existând numeroase metode pentru aceasta soluţie. AHO arată că problema calculării MCC este o problema "NP complete". Conform propoziţiei 1, reuniunea AMCC asociată clasei incompatibile MIC_4 este :

$$\bigcup_{MIC_4} AMCC=MCC_1 \cup MCC_3 \cup MCC_2 \cup MCC_6 \cup MCC_7$$

$$\bigcup_{MIC_4} =\{mO_1,mO_2,mO_3,mO_4,mO_5,mO_6,mO_7\}$$

Prop. 3

Dacă un set de microoperaţii MB(mO) este partiţionat în q câmpuri ale unei microinstrucţiuni, costul minim se va realiza atunci când (q-1) câmpuri specifică câte o microoperaţie (au un singur bit), iar cel de al q-lea câmp codifică restul de |MB(mO)|<-q+1 microoperaţii.

Dem.

Vom descrie o demonstraţie echivalentă. Vom arăta că pentru o lungime de microinstrucţiune dată LMI şi pentru un număr de câmpuri q dat se pot codifica maximum de microoperaţii, iar ultimul câmp are LMI-q+1 biţi. Considerăm o partiţie arbitrară a LMI biţi în q câmpuri. Fie câmpul cu b_{max} biţi, câmpul cu lungimea cea mai mare şi fie un oricare alt câmp care are b_i biţi. Numărul de microoperaţii ce poate fi codificat de cele două câmpuri este:

$$NMO = (2^{bmax}-1)+(2^{bi}-1)$$

Facem o modificare în organizarea logică a microinstrucţiunii şi mutăm un bit din câmpul cu b_i biţi în câmpul cu b_{max} biţi. în acest caz numărul de microoperaţii ce se pot codifica este:

$$NMO'= (2^{(bmax+1)}-1)+(2^{(bi-1)}-1)$$

$$NMO'-NMO= 2^{bmax}-2^{bi} \geq 0 \quad \text{deoarece } b_{max} \geq b_i.$$

Deci se pot codifica mai multe operaţii (microoperaţii) după modificarea structurii microinstrucţiunii.

Repetând procesul de mutare a unui bit dintr-un câmp oarecare în câmpul de lungime maximă, numărul de microoperaţii ce poate fi codificat creşte. Numărul maxim de microoperaţii ce poate fi codificat rezultă a fi egal cu $(q+2^{(LMI-q+1)}-2)$. Transformând totul în cost se obtine costul minim:

Cost = $q-1+[\log_2(|MB(mO)|-q+2)]$ când (q-1) câmpuri specifică fiecare câte o microoperaţie, iar ultimul câmp (|MB(mO)|-q+1) microoperaţii.

Prop. 4

Nu este posibilă o soluţie de partiţionare a setului MB(mO) microoperaţii în (q+h) câmpuri astfel încât costul să fie mai mic decât partiţia în q câmpuri.

Dem.

Fie

$C = q-1+[\log_2(|MB(mO)|-q+2)]$ costul minim obţinut prin partiţia în q câmpuri şi

$C = q+h-1+[\log_2(|MB(mO)|-q-h+2)]$ costul minim obţinut prin partiţia în q+h câmpuri.

Deosebim 2 cazuri şi anume:

1) $h=1$ şi $|MB(mO)|-q-1=2^k$

2) $h \neq 1$ sau $|MB(mO)|-q+1$ ą 2^k

în primul caz:

$C_q=q-1+k+1=q+k$

$C_{(q+h)}=q+k$

deci $C_q=C_{(q+h)}$

în al doilea caz, deoarece :

$[\log_2(|MB(mO)|-q+2)]-[\log_2(|MB(mO)|-q-h+2)] < h$

rezultă $C_q < C_{(q+h)}$

Observaţia 6

Costul minim pentru codificarea setului de microoperaţii MB(mO) poate să fie mai mare decât cel dat de propoziţia 3 adică:

$C \geq q-1+[\log_2(|MB(mO)|-q+2)]$ când $|MCC|_{max} \leq |MB(mO)|-q$.

într-adevăr costul minim corespunde când:

(q-1) câmpuri specifică fiecare câte o microoperaţie;

cel de-al q-lea câmp codifică |MB(mO)|-q+1 microoperaţii.

Dacă $|MCC|_{max} \leq |MB(mO)|-q$, rezultă că $|MCC|_{max}<|MB(mO)|-q+1$, ceea ce ar face ca în ultimul câmp să nu fie toate microoperaţiile compatibile.

În acest caz trebuie renunţat la organizarea microinstrucţiunii în (q-1) câmpuri de 1 bit, dar prin aceasta se măreşte şi costul de implementare.

În continuare vom prezenta o metodă de minimizare a numărului de biţi necesari pentru codificarea microinstrucţiunilor complete generate prin partiţia unui microbloc.

Etapele metodei

1. Se alege clasa de incompatibilitate maximă care are cardinalitatea maximă MIC. Fie

$MIC_m \in MIC$ astfel încât $|MIC_m| \geq |MIC_j|$ pentru $j \neq m$,

$1 \leq j \leq |MIC|$

Se generează clasele de compatibilitate maximă asociate AMCC clasei MIC.

$AMCC_m$={MCC≥ pt orice mO $\in MIC_m$ $\exists$ MCC astfel încât mO $\in$ MCC}

2. Se formează tabela de acoperire modificată prin considerarea numai a claselor de compatibilitate maximă ce aparţin AMCC.

TAM : $AMCC_m$ x MB(mO) ÄÄ> B

Se caută multimea de clase de compatibilitate maximă esenţiale {MCC_e} inclus în $AMCC_m$ astfel încât există MCC_e unic pentru care : $mO_i \in MCC_e$ avem TAM (MCC_e,mOi)=1.

Se elimină coloanele corespunzătoare microoperaţiilor ce sunt acoperite de clasele esenţiale şi cele corespunzătoare microoperaţiilor componente ale clasei MIC_m, obţinându-se o tabelă de acoperire redusă :

TAR : $AMCC_m$ x (MB(mO) \ MIC_m) \ {MCC_e} ÄÄ> B

3. Se generează setul soluţiilor de acoperire a microoperaţiilor {MCC_{ap}} : {MB(mO) \ MIC_m) \ {MCC_e}

Fie SOL={SOL_1,SOL_2,..SOL_p} soluţiile de acoperire în care

$SO_j = \wedge$ / ({MCC_e} $\cup$ {MCC_{ap}})

Se consideră soluţia parţială cu cardinalitatea minimă SOL_j.

Se generează soluţia de acoperire a microoperaţiilor ce aparţin clasei MIC_m, încă neacoperite: {MCC_{am}}.

4. Se generează setul soluţiilor de acoperire completă:

$SOLC_j$=SOL_j ^ {MCC_{am}}.

Se calculează costul soluţiei de acoperire completă cu cardinalitatea minimă.

Dacă costul nu este minim (sau aproape minim, având în vedere şi consideraţiile de ordin fizic ale componentelor memoriei de control), se alege o altă solutie de acoperire completă.

În cazul în care, toate soluţiile complete asociate unei soluţii parţiale nu au cost minim, se alege o nouă soluţie parţială pentru care se repetă etapa a 4-a.

Metoda propusă pentru codificarea microinstrucţiunilor complete nu va genera o soluţie optimă, însă va genera o soluţie într-un timp rezonabil.

Pentru multe probleme practice, soluţia asigură o partiţie optimă. De notat că, dacă numărul de biţi obţinut pentru codificarea microinstrucţiunilor este egal cu (4n+1) sau (8n+1) acesta se poate optimiza astfel încât să se obtină 4n sau 8n. Această optimizare este necesară datorită faptului că, în general, componentele fizice sunt organizate pe 4 sau 8 biţi. Optimizarea propusă se poate face pe seama paralelismului, introducând relaţii de ordine parţială suplimentare în graful de dependenţă de date asociat microsubblocului.

Exemplul 2

Considerăm setul de microinstrucţiuni din cadrul microsubblocului :

MB={mO_1,mO_2,mO_3,mO_4,mO_5,mO_6,mO_7,mO_8,mO_9,mO_{10}}

Presupunem că pe baza dependenţei de date şi a conflictului de resurse între microoperaţii a rezultat următoarea partiţie a microsubblocului :

mIC_1={mO_1,mO_2,mO_4}

mIC_2={mO_1,mO_3,mO_5}

mIC_3={mO_2,mO_6,mO_8,mO_9}

$mIC_4=\{mO_4,mO_5,mO_7,mO_8\}$
$mIC_5=\{mO_3,mO_4,mO_5,mO_6,mO_7\}$
$mIC_6=\{mO_6,mO_9,mO_{10}\}$
$mIC_7=\{mO_7,mO_{10}\}$

Tabela de incompatibilitate este :

mO\ mO	mO_1	mO_2	mO_3	mO_4	mO_5	mO_6	mO_7	mO_8	mO_9	mO_{10}
m_1	x	x	x	x	x					
m_2	x	x		x		x		x	x	
m_3	x		x	x	x	x	x			
m_4	x	x	x	x	x	x	x	x		
m_5	x		x	x	x	x	x	x		
m_6		x	x	x	x	x	x	x	x	x
m_7			x	x	x	x	x	x		x
m_8		x		x	x	x	x	x	x	
m_9		x				x		x	x	x
m_{10}						x	x		x	x

Clasele maximale de compatibilitate sunt :

$MIC_1=\{mO_1,mO_2,mO_4\}$
$MIC_2=\{mO_1,mO_3,mO_4,mO_5\}$
$MIC_3=\{mO_2,mO_4,mO_6,mO_8\}$
$MIC_4=\{mO_2,mO_6,mO_8,mO_9\}$
$MIC_5=\{mO_3,mO_4,mO_5,mO_6,mO_7\}$
$MIC_6=\{mO_4,mO_5,mO_6,mO_7,mO_8\}$
$MIC_7=\{mO_6,mO_7,mO_{10}\}$
$MIC_8=\{mO_6,mO_9,mO_{10}\}$

Clasele maximale de compatibilitate sunt :

$MCC_1=\{mO_1,mO_6\}$
$MCC_2=\{mO_1,mO_7,mO_9\}$
$MCC_3=\{mO_1,mO_8,mO_{10}\}$
$MCC_4=\{mO_2,mO_3,mO_{10}\}$
$MCC_5=\{mO_2,mO_5,mO_{10}\}$
$MCC_6=\{mO_2,mO_7\}$
$MCC_7=\{mO_3,mO_8,mO_{10}\}$
$MCC_8=\{mO_3,mO_9\}$
$MCC_9=\{mO_4,mO_9\}$
$CC_{10}=\{mO_4,mO_10\}$
$CC_{11}=\{mO_5,mO_9\}$

Tabela de acoperire a microoperaţiilor de către clasele maximale de compatibilitate este :

MCC\mO	mO_1	mO_2	mO_3	mO_4	mO_5	mO_6	mO_7	mO_8	mO_9	mO_{10}
MCC_1	x					x				

MCC_2	x						x		x	
MCC_3	x							x		x
MCC_4		x	x							x
MCC_5		x			x					x
MCC_6		x					x			
MCC_7			x					x		x
MCC_8			x						x	
MCC_9				x					x	
MCC_{10}				x						x
MCC_{11}					x				x	

Se observă că există două clase maximale de incompatibilitate cu cardinalitate 5 (MIC_5 şi MIC_6).

Costul minim absolut, conform proprietăţii 4 este **C = 7**.

Să considerăm $AMCC_6$ - clasele maximale de compatibilitate asociate clasei maximale de incompatibilitate MIC_6 :

$AMCC_6=\{MCC_1,MCC_2,MCC_3,MCC_5,MCC_6,MCC_7,MCC_9,MCC_{10},MCC_{11})$

Conform propoziţiei 1 clasele maximale de compatibilitate din $AMCC_6$ acoperă toate microoperaţiile microsubblocului.

Tabela de acoperire modificată este :

	mO_1	mO_2	mO_3	mO_4	mO_5	mO_6	mO_7	mO_8	mO_9	mO_{10}
MCC_1	x					**x**				
MCC_2	x						x		x	
MCC_3	x							x		x
MCC_5		x			x					x
MCC_6		x					x			
MCC_7			**x**					x		
MCC_9				x					x	
MCC_{10}				x						x
MCC_{11}					x				x	

Se observă că MCC_1 şi MCC_7 sunt esenţiale şi vor face parte din soluţia finală. Considerăm, conform algoritmului, acoperite microoperaţiile incluse în clasele de compatibilitate maximă MCC_1 şi MCC_7 esentiale şi cele componente ale clasei de incompatibilitate maximă MIC_6. Astfel, din tabela de acoperire se elimină microoperaţiile $mO_1,mO_6,mO_3,mO_8,mO_{10}$, respectiv mO_4,mO_5,mO_7.

Tabela de acoperire redusă este :

MCC \mO	mO_2	mO_9
MCC_2		x
MCC_5	x	
MCC_6	x	
MCC_9		x
MCC_{11}		x

Se observă că tabela de acoperire s-a redus substanţial. Acoperirea microoperaţiilor mO_2 şi mO_9 se poate face cu ajutorul următoarelor clase maximale de compatibilitate :

MCC_2,MCC_5 MCC_2,MCC_6 MCC_5,MCC_9 MCC_5,MCC_{11} MCC_6,MCC_9 MCC_6,MCC_{11}

Se poate forma setul soluţiilor :

$SOL = \{SOL_1,SOL_2,SOL_3,SOL_4,SOL_5,SOL_6\}$ astfel :

$SOL = \{MCC_1\ MCC_7\ MCC_2\ MCC_5$; $MCC_1\ MCC_7\ MCC_2\ MCC_6$;
$MCC_1\ MCC_7\ MCC_5\ MCC_9$; $MCC_1\ MCC_7\ MCC_5\ MCC_{11}$;
$MCC_1\ MCC_7\ MCC_6\ MCC_9$; $MCC_1\ MCC_7\ MCC_6\ MCC_{11}\}$

Considerând soluţia : $MCC_1\ MCC_7\ MCC_5\ MCC_9 \in SOL$, se acoperă toate microoperaţiile cu excepţia $mO_7 \in MIC_6$. Pentru acoperirea lui mO_7 se poate lua una din clasele maximale de compatibilitate MCC_2 sau MCC_6.

Astfel setul soluţiilor complete este :

$SOLC_3 = \{SOLC_{31},SOC_{32}\}$

unde

$SOLC_{31}=\{MCC_1,MCC_7,MCC_5,MCC_9,MCC_2\}$
$SOLC_{32}=\{MCC_1,MCC_7,MCC_5,MCC_9,MCC_6\}$

Ambele soluţii complete au aceeaşi cardinalitate ce specifică numărul minim de câmpuri necesar pentru codificarea microinstrucţiunilor ce descriu microsubblocul.

Alegând $SOLC_{31}$, rezultă următoarea grupare a microoperaţiilor:

$\{ mO_1\ mO_7\ mO_9$; $mO_2\ mO_5$; $mO_3\ mO_8\ mO_{10}$; mO_4 ; $mO_6 \}$

ce necesită 8 biţi pentru codificare.

Soluţia este acceptabilă deoarece are un cost apropiat de costul minim absolut.

Costul minim al acestei probleme, calculat prin metoda prezentată de Das şi Banerjee este tot 8, însă numărul de operaţii desfăşurat de algoritmul descris de ei este substanţial mai mare. Prin procedura prezentată se pot calcula soluţii pentru multe probleme, însă în general, se stabilesc soluţii aproape minime. Avantajul constă în faptul că rezolvarea se face într-un timp polinomial.